

Flight Booking Website — Explanation (Part 5/5)

Part 5: End-to-end user flows, data flow, and known mismatches.

1) End-to-end flow (Passenger)

1. **Register / Login:** Passenger uses `register.html` and `login.html` to create/login. Backend: `backend/api/register.php`, `backend/api/auth.php`.
2. **Search flights:** Passenger selects from/to cities. Backend: `backend/api/search-flights.php` (World A) or a seat-class search endpoint (World B).
3. **View flight:** Passenger opens a flight page (details). Backend might call a “flight details” endpoint or reuse list info.
4. **Book:** Passenger creates a booking via `backend/api/bookings.php` with payment type (cash/account).
5. **Dashboard:** Passenger views bookings via `backend/api/passenger-bookings.php`.
6. **Cancel booking:** Passenger cancels; backend refunds if paid with account and updates flight counters.
7. **Messages:** Passenger contacts company via `backend/api/messages.php`.

2) End-to-end flow (Company)

1. **Register / Login:** Company uses the same register/login pages with `user_type=company`.
2. **Create flight:** Company adds a flight (fees, max passengers, etc.) and defines itinerary cities in sequence order.
3. **Company dashboard:** Lists company flights via `backend/api/company-flights.php`.
4. **Flight details:** Shows pending + registered passengers via `backend/api/company-flight-details.php`.
5. **Cancel flight:** Company cancels; backend cancels bookings and refunds account payments via `backend/api/cancel-flight.php`.
6. **Messages:** Company responds via `backend/api/messages.php`.

3) How data flows between frontend and backend

- Frontend builds a request (URL + JSON body) using `fetch()`.
- Backend reads `php://input` (JSON) or query params, validates them, and runs SQL.
- Backend returns JSON; frontend updates DOM with the result or shows errors.
- User/session state is stored client-side (browser storage).

4) Known mismatches / gotchas (very important for students)

4.1 Schema vs models mismatch

The schema in `database/project_schema.sql` (World A) does not match the expectations of the PHP model classes in `backend/models` (World B). Example mismatch patterns:

- Models expect airports/airlines tables; `project_schema` doesn't define them.
- Models expect seat availability by class; `project_schema` uses max/pending/registered counters.
- Some endpoints are written directly against World A tables without using models.

4.2 Frontend API wrapper vs endpoints mismatch

`frontend/js/api.js` may call endpoints with names/parameters aligned with World B while dashboards call World A endpoints. When debugging, always confirm the exact endpoint URL used by that page.

4.3 Security assumptions (project-level)

- Many endpoints accept user ids in requests without authenticating them. This is fine for a demo, but not secure for real systems.
- File uploads (profile images/passports/logos) need server-side validation to be safe in production.

5) Suggested “student split” for teaching (same 5 parts)

1. Part 1: Overview + Database
2. Part 2: Backend Config + Auth
3. Part 3: Backend APIs (flights/bookings/messages)
4. Part 4: Frontend (pages + JS + CSS)
5. Part 5: End-to-end flows + debugging mismatches

6) Files covered in Part 5

- All remaining “glue”: how parts connect across backend/api and frontend
- Highlights the schema/endpoint inconsistencies so students can run the right path

End of Part 5/5.